Sim2Real Predictivity: Does Evaluation in Simulation Predict Real-World Performance?

Abhishek Kadian, Joanne Truong, Aaron Gokaslan, Alexander Clegg, Erik Wijmans, Stefan Lee, Manolis Savva, Sonia Chernova, and Dhruv Batra

***Abstract*—Does progress in simulation translate to progress on robots? If one method outperforms another in simulation, how likely is that trend to hold in reality on a robot? We examine this question for embodied PointGoal navigation – developing engineering tools and a research paradigm for evaluating a simulator by its *sim2real predictivity*. First, we develop Habitat-PyRobot Bridge (HaPy), a library for seamless execution of identical code on simulated agents and robots – transferring simulation-trained agents to a LoCoBot platform with a one-line code change. Second, we investigate the sim2real predictivity of Habitat-Sim M. Savva *et al.*, for PointGoal navigation. We 3D-scan a physical lab space to create a *virtualized replica*, and run parallel tests of 9 different models in reality and simulation. We present a new metric called Sim-vs-Real Correlation Coefficient (SRCC) to quantify predictivity. We find that SRCC for Habitat as used for the CVPR19 challenge is low (0.18 for the success metric), suggesting that performance differences in this simulator-based challenge do not persist after physical deployment. This gap is largely due to AI agents learning to exploit simulator imperfections – abusing collision dynamics to 'slide' along walls , leading to shortcuts through otherwise non-navigable space. Naturally, such exploits do not work in the real world. Our experiments show that it is possible to *tune* simulation parameters to improve sim2real predictivity (e.g. improving $SRCC_{Succ}$ from 0.18 to 0.844) – increasing confidence that in-simulation comparisons will translate to deployed systems in reality.**

***Index Terms*—Visual-based navigation, reinforcement learning.**

I. Introduction

All simulations are wrong, but some are useful.

A variant of a popular quote by George Box

Manuscript received February 24, 2020; accepted June 30, 2020. Date of publication August 4, 2020; date of current version August 26, 2020. This letter was recommended for publication by Associate Editor E. Ricci and Editor E. Marchand upon evaluation of the Reviewers' comments. The Georgia Tech effort was supported in part by NSF, AFRL, DARPA, ONR YIPs, ARO PECASE, Amazon. *(Corresponding author: Abhishek Kadian.)*

Abhishek Kadian, Aaron Gokaslan, and Alexander Clegg are with Facebook AI Research, Menlo Park, CA 94025 USA (e-mail: akadian@fb.com; agokaslan@fb.com; alexclegg@fb.com).

Joanne Truong is with Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: truong.j@gatech.edu).

Erik Wijmans, Sonia Chernova, and Dhruv Batra are with Facebook AI Research, Menlo Park, CA 94025 USA, and also with Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: erikwijmans@fb.com; schernova@fb.com, chernova@gatech.edu; dbatra@fb.com, dbatra@gatech.edu).

Stefan Lee is with Oregon State University, Corvallis, OR 97331 USA (e-mail: leestef@oregonstate.edu).

Manolis Savva is with Facebook AI Research, Menlo Park, CA 94025 USA, and also with Simon Fraser University, Vancouver, V5A 1S6, Canada (e-mail: msavva@fb.com).

This letter has supplementary downloadable material available at https://ieeexplore.ieee.org, provided by the authors.

Digital Object Identifier 10.1109/LRA.2020.3013848

THE VISION, language, and learning communities have recently witnessed a resurgence of interest in studying integrative robot-inspired agents that perceive, navigate, and interact with their environment. For a variety of reasons, such work has commonly been carried out in simulation rather than in real-world environments. Simulators can run orders of magnitude faster than real-time [1], can be highly parallelized, and enable *decades* of agent experience to be collected in days [2]. Moreover, evaluating agents in simulation is safer, cheaper, and enables easier benchmarking of scientific progress than running robots in the real-world. Consequentially, these communities have rallied around simulation as a testbed – developing several increasingly realistic indoor/outdoor navigation simulators [1], [3]–[9], designing a variety of tasks set in them [3], [10], [11], holding workshops about such platforms [12], and even running challenges in these simulated worlds [13]–[15]. As a result, significant progress has been made in these settings. For example, agents can reach point goals in novel home environments with near-perfect efficiency [2], control vehicles in complex, dynamic city environments [4], follow natural-language instructions [10], and answer questions [11].

However, no simulation is a perfect replica of reality, and AI systems are known to exploit imperfections and biases to achieve strong performance in simulation which may be unrepresentative of performance in reality. Notable examples include evolving tall creatures for locomotion that fall and somersault instead of learning active locomotion strategies [16] and OpenAI's hide-and-seek agents abusing their physics engine to 'surf' on top of obstacles [17].

This raises several fundamental questions of deep interest to the scientific and engineering communities: **Do comparisons drawn from simulation translate to reality for robotic systems?** Concretely, if one method outperforms another in simulation, will it continue to do so when deployed on a robot? Should we trust the outcomes of embodied AI challenges (e.g. the AI Habitat Challenge at CVPR 2019) that are performed entirely in simulation? These are questions not only of simulator *fidelity*, but rather of *predictivity*.

In this work, we examine the above questions in the context of visual navigation – focusing on measuring and optimizing the predictivity of a simulator. High predictivity enables researchers to use simulation for evaluation with confidence that the performance of different models will generalize to real robots. Given this focus, our efforts are orthogonal to techniques for sim2real transfer, including those based on adjusting simulator

2377-3766 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.

parameters. To answer these questions, we introduce engineering tools and a research paradigm for performing simulation-to-reality (sim2real) indoor navigation studies, revealing surprising findings about prior work.

First, we develop the Habitat-PyRobot Bridge (HaPy), a software library that enables seamless sim2robot transfer. HaPy is an interface between (1) Habitat [1], a high-performance photorealistic 3D simulator, and (2) PyRobot [18], a high-level python library for robotics research. Crucially, HaPy makes it trivial to *execute identical code in simulation and reality*. Sim2robot transfer with HaPy involves only a single line edit to the code (changing the config.simulator variable from Habitat-Sim-v0 to PyRobot-Locobot-v0), essentially treating reality as just another simulator! This reduces code duplication, provides an intuitive high-level abstraction, and allows for rapid prototyping with modularity (training a large number of models in simulation and 'tossing them over' for testing on the robot). In fact, all experiments in this article were conducted by a team of researchers physically separated by thousands of miles – one set training and testing models in simulation, another conducting on-site tests with the robot, made trivial due to HaPy. We will open-source HaPy so that everyone has this ability.

Second, we propose a general experimental paradigm for performing sim2real studies, which we call *sim2real predictivity*. Our thesis is that simulators need not be a perfect replica of reality to be useful. Specifically, we should primarily judge simulators not by their visual or physical realism, but by their *sim2real predictivity* – if method A outperforms B in simulation, how likely is the trend to hold in reality? To answer this question, we propose the use of a quantity we call Sim2Real Correlation Coefficient (SRCC).

We prepare a real lab space within which the robot must navigate while avoiding obstacles. We then *virtualize* this lab space (under different obstacle configurations) by 3D scanning the space and importing it in Habitat. Armed with the power to perform parallel trials in reality and simulation, we test a suite of navigation models both in simulation and in the lab with a real robot. We then produce a scatter plot where every point is a navigation model, the x-axis is the performance in simulation, and the y-axis is performance in reality. SRCC is shown in a box at the top. If SRCC is high (close to 1), this is a 'good' simulator setting in the sense that we can conduct scientific development and testing purely in simulation, with confidence that we are making 'real' progress because the improvements in simulation will generalize to real robotic testbeds. If SRCC is low (close to 0), this is a 'poor' simulator, and we should have no confidence in results reported solely in simulation. We apply this methodology in the context of PointGoal Navigation (PointNav) [19] with Habitat and the LoCoBot robot [20] as our simulation and reality platforms – our experiments made easy with HaPy. These experiments reveal a number of surprising findings:

1) We find that SRCC for Habitat as used for the CVPR19 challenge is 0.603 for the Success weighted by (normalized inverse) Path Length (SPL) metric and 0.18 for agent success. When ranked by SPL, we observe 9 relative ordering reversals from simulation to reality, suggesting that the results/winners may not be the same if the challenge were run on LoCoBot.
2) We find that large-scale RL trained models can learn to 'cheat' by exploiting the way Habitat allows for 'sliding' along walls on collision. Essentially, the virtual robot is capable of cutting corners by sliding around obstacles, leading to unrealistic shortcuts through parts of non-navigable space and 'better than optimal' paths. Naturally, such exploits do not work in the real world where the robot stops on contact with walls.
3) We *optimize* SRCC over Habitat design parameters and find that a few simple changes improve SRCC_{SPL} from 0.603 to 0.875 and $\text{SRCC}_{\text{Succ}}$ from 0.18 to 0.844. The number of rank reversals nearly halves to 5 (13.8%). Furthermore, we identify highly-performant agents in *both* this new simulation and on LoCoBot in real environments.

While our experiments are conducted on the PointNav task, we believe our software (HaPy), experimental paradigm (sim2real predictivity and SRCC), and take-away messages are useful to the broader community. While we believe our controlled environment is complex enough to robustly estimate sim2real predictivity, we do not believe it should be used to measure navigation performance. Our work is complementary to ongoing efforts to improve sim2real performance and our metric is independent of the simulator implementation.

II. Related Work

Embodied AI tasks: Given the emergence of several 3D simulation platforms, it is not surprising that there has been a surge of research activity focusing on investigation of embodied AI tasks. One early example leveraging simulation is the work of Zhu *et al.* [21] on target-driven navigation using deep reinforcement learning in synthetic environments within AI2 THOR [7]. Follow up work by Gupta *et al.* [22] demonstrated an end-to-end learned joint mapping and planning method evaluated in simulation using reconstructed interior spaces. More recently, Gordon *et al.* [23] showed that decoupling perception and policy learning modules can aid in generalization to unseen environments, as well as between different environment datasets. Beyond these few examples, a breadth of recent work on embodied AI tasks demonstrates the acceleration that 3D simulators have brought to this research area. In contrast, deployment on real robotic platforms for similar AI tasks still incurs significant resource overheads and is typically only feasible with large, well-equipped teams of researchers. One of the most prominent examples is the DARPA Robotics Challenge (DRC) [24]. Another example of real-world deployment is the work of Gandhi *et al.* [25] who trained a drone to fly in reality by locking it in a room. Our goal is to characterize how well a model trained in simulation can generalize when deployed on a real robot.

Simulation-to-reality transfer: Due to the logistical limitations of physical experimentation, transfer of agents trained in simulation to real platforms is a topic of much interest. There have been successful demonstrations of sim2real transfer in several domains. The CAD2RL [8] system of Sadeghi and

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.

Levine trained a collision avoidance policy entirely in simulation and deployed it on real aerial drones. Similarly, Muller *et al.* [26] show that driving policies can be transferred from simulated cars to real remote-controlled cars by leveraging modularity and abstraction in the control policy. Tan *et al.* [27] train quadruped locomotion policies in simulation by leveraging domain randomization and demonstrate robustness when deployed to real robots. Chebotar *et al.* [28] improve the simulation using the difference between simulation and reality observations. Lastly, Hwangbo *et al.* [29] train legged robotic systems in simulation and transfer the learned policies to reality. The goal of this work is to enable researchers to use simulation for evaluation with confidence that their results will generalize to real robots. This brings to the forefront the key question: can we establish a correlation between performance in simulation and in reality? We focus on this question in the domain of indoor visual navigation.

III. Habitat-PyRobot Bridge: Simple Sim2Real

Deploying AI systems developed in simulation to physical robots presents significant financial, engineering, and logistical challenges – especially for non-robotics researchers. Approaching this directly requires researchers to maintain two parallel software stacks, one typically based on ROS [30] for the physical robot and another for simulation. In addition to requiring significant duplication of effort, this model can also introduce inconsistencies between agent details and task specifications in simulation and reality.

To reduce this burden and enable our experiments, we introduce the Habitat-PyRobot Bridge (HaPy). As its name suggests, HaPy integrates the Habitat [1] platform with PyRobot APIs [18] – enabling identical agent and evaluation code to be executed in simulation with Habitat and on a PyRobot-enabled physical robot. Habitat is a platform for embodied AI research that aims to standardize the different layers of the embodied agent software stack, covering 1) datasets, 2) simulators, and 3) tasks. This enables researchers to cleanly define, study, and share embodied tasks, metrics, and agents. For deploying simulation-trained agents to reality, we replace the simulator layer in this stack with 'reality' while maintaining task specifications and agent interfaces. Towards this end, we integrate Habitat with PyRobot [18], a recently released high-level API that implements simple interfaces to abstract lower-level control and perception for mobile platforms (LoCoBot), and manipulators (Sawyer), and offers a seamless interface for adding new robots. HaPy is able to benefit from the scalability and generalizability of both Habitat and PyRobot. Running an agent developed in Habitat on the LoCoBot platform requires changing a single argument, `Habitat-Sim-v0→PyRobot-Locobot-v0`.

At a high level, HaPy enables the following:

1) A uniform observation space API across simulation and reality. Having a shared implementation ensures that observations from simulation and reality sensors go through the same transformations (e.g. resizing, normalization).
2) A uniform action space API for agents across simulation and reality. Habitat and PyRobot differ in their agent action spaces. Our integration unifies the two action spaces and allows an agent model to remain agnostic between simulation and reality.
3) Integration at the simulator layer in the embodied agent stack allows reuse of functionalities offered by task layers of the stack – task definition, metrics, etc. stay the same across simulation and reality. This also opens the potential for *jointly* training in simulation and reality.
4) Containerized deployment for running challenges where participants upload their code for seamless deployment to mobile robot platforms such as LoCoBot.

We hope this contribution will allow the community to easily build agents in simulation and deploy them in the real world.

IV. Visual Navigation in Simulation & Reality

Recall that our goal is to answer an ostensibly straightforward scientific question – is performance in simulated environments predictive of real-world performance for visual navigation? Let us make this more concrete.

First, note this question is about *testing* in simulation vs reality. It does not require us to take a stand on *training* in simulation vs reality (or the need for training at all). For a comparison between simulation-trained and non-learning-based navigation methods, we refer the reader to previous studies [1], [31], [32]. We focus on test-time discrepancies between simulation and reality for learning-based methods.

Second, even at test-time, many variables contribute to the sim2real gap. The real-world test environment may include objects or rooms which visually differ from simulation, or may present a differing task difficulty distribution (due to unmodeled physics or rendering in simulation). To isolate these factors as much as possible, we propose a direct comparison – evaluating agents in physical environments and in corresponding simulated replicas. We construct a set of physical lab environment configurations for a robot to traverse and virtualize each by 3D scanning the space, thus controlling for semantic domain gap. We then evaluate agents in matching simulated and real configurations to characterize the sim-vs-real gap in visual navigation.

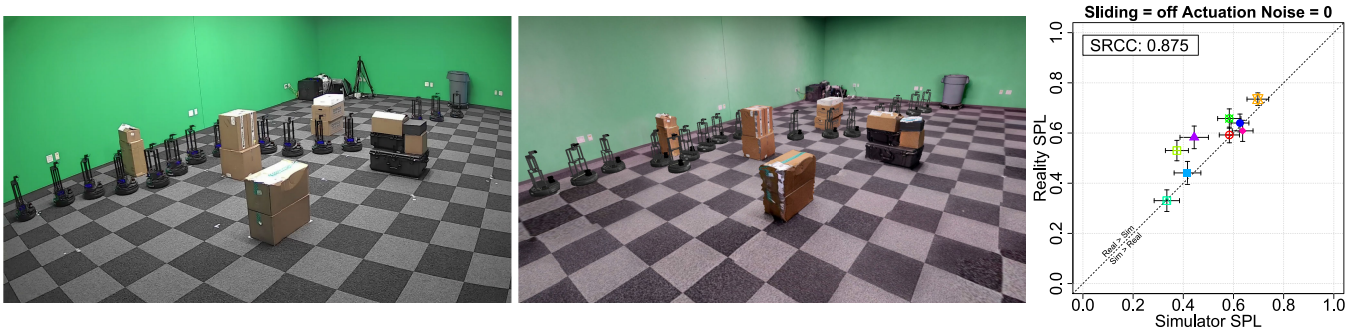
In this section, we first recap the PointNav task [19] from the recent AI Habitat Challenge [13]. Then, we compare agents in simulation and robots in reality.

A. Task: PointGoal Navigation (PointNav)

In this task, an agent is spawned in an unseen environment and asked to navigate to a goal location specified in relative coordinates. We start from the agent specification and observation settings from [13]. Specifically, agents have access to an egocentric RGB (or RGBD) sensor and accurate localization and heading via a GPS+Compass sensor. The goal is specified using polar coordinates (r, θ), where r is the Euclidean distance to the goal and θ is the azimuth to the goal. The action space for the agent is: `turn-left 30°`[1], `turn-right 30°`[1], `forward 0.25 m`, and `STOP`. An episode is considered successful if the agent issues the `STOP` command within 0.2 meters of the goal. Episodes

[1]Originally 10°.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.



(a) Reality. (b) Simulation. (c) Sim-vs-Real Correlation.

Fig. 1. We measure the correlation between visual navigation performance in simulation and in reality by virtualizing reality and executing parallel experiments. (a) Navigation trajectory in a real space with obstacles. (b) virtualized replica in simulation. (c) we propose the Sim2Real Correlation Coefficient (SRCC) as a measure of simulation predictivity. By optimizing for SRCC, we arrive at simulation settings that are highly predictive of real-world performance.

lasting longer than 200 steps or calling the STOP command $>$0.2 m from goal are declared unsuccessful. We set the step threshold to 200 (compared to 500 in the challenge) because our testing lab is small and we found that episodes longer than 200 actions are likely to fail. We also limit collisions to 40 to prevent damage to the robot. We find that $>$40 collisions in an episode typically occur when the robot is stuck and likely to fail.

B. Agent in Simulation

Body: The experiments by Savva *et al.* [1] and the Habitat Challenge 2019 [13] model the agent as an idealized cylinder of radius 0.1m and height 1.5$\tilde{m}$. As shown in Fig. 3, we configure the agent to match the robot used in our experiments (LoCoBot) as closely as possible. Specifically, we configure the simulated agent's base-radius and height to be 0.175 m and 0.61 m respectively to match LoCoBot dimensions.

Sensors: We set the agent camera field of view to 45 degrees to match images from the Intel D435 camera on LoCoBot. We match the aspect ratio and resolution of the simulated sensor frames to real sensor frames from LoCoBot using square center cropping followed by image resizing to a height and width of 256 pixels. To mimic the depth camera's limitations, we clip simulated depth sensing to 10 m.

Actions: In [1], [13], agent actions are deterministic – *i.e.* when the agent executes turn-left 30°, it turns *exactly* 30°, and forward 0.25 m moves the agent *exactly* 0.25 m forward (modulo collisions). However, no robot moves deterministically due to real-world actuation noise. To model the actions on LoCoBot, we leverage an actuation noise model derived from mocap-based benchmarking by the PyRobot authors [18]. Specifically, when the agent calls (say) forward, we sample from an action-specific 2D Gaussian distribution over relative displacements. Fig. 2 shows trajectory rollouts sampled from this noise model. As shown, identical action sequences can lead to vastly different final locations.

Finally, in contrast to [1], [13], we increase the angles associated with turn-left and turn-right actions from 10° to 30° degrees. The reason is a fundamental discrepancy between simulation and reality – there is no 'ideal' GPS+compass sensor

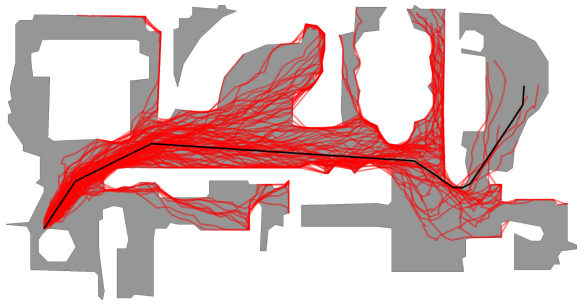


Fig. 2. Effect of actuation noise. The black line is a trajectory from an action sequence with perfect actuation. In red are trajectories from this sequence with actuation noise.

in reality. Perfect localization in indoor environments is an open research problem. In our preliminary experiments, we found that localization noise was exacerbated by the 'move, stop, turn, move' behavior of the robot, which is a result of a discrete action space (as opposed to continuous control via velocity or acceleration actions). We strike a balance between staying comparable to prior work (that uses discrete actions) and reducing localization noise by increasing the turn angles (which decreases the number of robot restarts). In the longer term, we believe the community should move towards continuous control to overcome this issue. All the modeling parameters are easily adaptable to different robots.

C. LoCoBot in Reality

Body: LoCoBot is designed to provide easy access to a robot with basic grasping, locomotion, and perception capabilities. It is a modular robot based on a Kobuki YMR-K01-W1 mobile base with an extensible body.

Sensors: LoCoBot is equipped with a Intel D435 RGB+depth camera. While LoCoBot possesses on-board IMUs and motor encoders (which can provide the GPS+Compass sensor observations required by this task), the frequent stopping and starting from our discrete actions resulted in significant error accumulation. To provide precise localization, we mounted a Hokuyo UTM-30LX LIDAR sensor in place of the robot's

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.

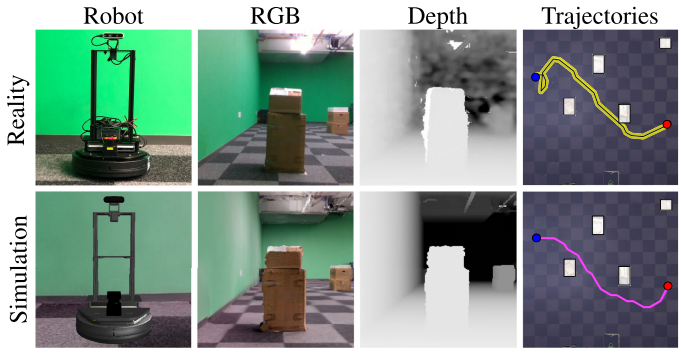


Fig. 3. Simulation vs. reality. Shading on the trajectory in reality represents uncertainty in the robot's location (±7 cm).

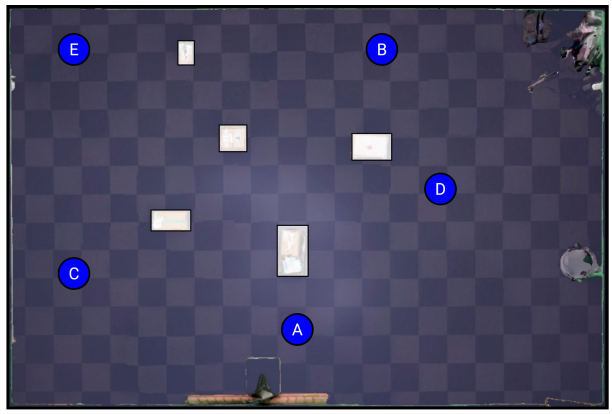


Fig. 4. Top-down view of one of our testing environments. White boxes are obstacles. The robot navigates sequentially through the waypoints $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$.

grasping arm (seen in Fig. 3). We run the LIDAR-based Hector SLAM [33] algorithm to provide the location+heading for the GPS+Compass sensor and for computing success and SPL of tests in the lab.

At this point, it is worth asking how accurate the LIDAR based localization is. To quantify localization error, we ran a total of 45 tests across 3 different room configurations in the lab, and manually measured the error with measuring tape. On average, we find errors of approximately 7 cm with Hector SLAM, compared to 40 cm obtained from wheel odometry and onboard IMU (combined with an Extended Kalman Filter implementation in ROS). Note that 7 cm is significantly lower than the 0.2 m = 20 cm criteria used to define success in PointNav, providing us confidence that we can use LIDAR-based Hector SLAM to judge success in our real-world experiments. More importantly, we notice that the LIDAR approach allows the robot to reliably relocalize using its surroundings, and thus error does not accumulate over long trajectories, or with consecutive runs, which is important for running hundreds of real-world experiments.

D. Evaluation Environment

Our evaluation environment is a controlled 6.5 m by 10 m interior room called CODA. Note that the agent was trained entirely in simulation and has never seen our evaluation environment room during training. The grid-like texture of the floor is purely coincidental, and not relied upon by the agent for navigation. We create 3 room configurations (easy, medium, hard difficulty) with increasing number of tall, cardboard obstacles spread throughout the room. These 'boxy' obstacles can be sensed easily by both the LIDAR and camera despite the two being vertically separated by ∼0.5 m. Objects like tables or chairs have narrow support at the LIDAR's height. For each room configuration, we define a set of 5 waypoints to serve as the start and end locations for navigation episodes. Fig. 4 shows top-down views of these room configurations.

Virtualization: We digitize each environment configuration using a Matterport Pro2 3D camera to collect 360° scans at multiple points in the room, ensuring full coverage. These scans are used to reconstruct 3D meshes of the environment which can be directly imported into Habitat. This streamlined process is easily scalable and enables quick virtualization of new physical spaces. On average, each configuration was reconstructed from 7 panoramic captures and took approximately 10 minutes. We also evaluated how the reconstruction quality in simulation affects the transfer, this was done by dropping 5% of mesh triangles in simulation which led to a 23% drop in SRCC (defined in Sec. IV.E).

Test protocol: We run parallel episodes in both simulation and reality. The agent navigates sequentially through the waypoints shown in Fig. 4($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$) for a total of 5 navigation episodes per room configuration. The starting points, starting rotations, and goal locations are identical across simulation and reality. In total, we test 9 navigation models (described in the next section), in 3 different room configurations, each with 5 spawn-to-goal waypoints, and 3 independent trials, for a total of 810 runs in simulation and reality combined. Each spawn-to-goal navigation with LoCoBot takes approximately 6 minutes, corresponding to 40.5 hours of real-world testing. Safety guidelines require that a human monitor the experiments and at 8 hours a day, these experiments would take 5 days. With such long turn-around times, it is essential that we use a robust pipeline to automate (or semi-automate) our experiments and reduce the cognitive load on the human supervisor. After each episode, the robot is automatically reset and has no knowledge from its previous run. For unsuccessful episodes, the robot uses a prebuilt environment map to navigate to the next episode start position. The room is equipped with a wireless camera to remotely track the experiments. In future, we plan to connect this automated evaluation setup to a docker2robot challenge, where participants can push code to a repository, which is then automatically evaluated on a real robot in this lab environment.

E. Sim2Real Correlation Coefficient

To quantify the degree to which performance in simulation translates to performance in reality, we use a measure we call Sim2Real Correlation Coefficient (SRCC). Let (s_i, r_i) denote accuracy (episode success rate, SPL [10], etc.) of navigation method i in simulation and reality respectively.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.

Given a paired dataset of accuracies for n navigation methods $\{(s_1, r_1), \ldots, (s_n, r_n)\}$, SRCC is the sample Pearson correlation coefficient (bivariate correlation).[2]

SRCC values close to $+1$ indicate high linear correlation and are desirable, insofar as changes in simulation performance metrics correlate highly with changes in reality performance metrics. Values close to 0 indicate low correlation and are undesirable as they indicate changes of performance in simulation is not predictive of real world changes in performance. Note that this definition of SRCC also suggests an intuitive visualization: by plotting performance in reality against performance in simulation as a scatterplot we can reveal the existence of performance correlation and detect outlier simulation settings or evaluation scenarios.

Beyond the utility of SRCC as a simulation predictivity metric, we can also view it as an optimization objective for simulation parameters. Concretely, let θ denote parameters controlling the simulator (amount of actuation noise, lighting, etc.). We can view simulator design as optimization problem: $\max_\theta \text{SRCC}(S_n(\theta), R_n)$ where $S_n(\theta) = \{s_1(\theta), \ldots, s_n(\theta)\}$ is the set of accuracies in simulation with parameters θ and R_n is the same performance metric computed on equivalent episodes in reality. Note that θ affects performance in simulation $S_n(\theta)$ but not R_n since we are only changing test-time parameters. The specific navigation models themselves are held fixed. Overall, this gives us a formal approach to simulator design instead of operating on intuitions and qualitative assessments.

In contrast, if a simulator has low SRCC but high mean real world performance, researchers will not be able to use this simulator to make decisions (e.g. model selection) because they cant know if changes to performance in simulation will have a positive or negative effect on real-world performance. Every change will have to be tested on the physical robot.

V. Measuring the Sim2Real Gap

Navigation Models: We experiment with learning-based navigation models. Specifically, we train for PointGoal in Habitat on the 72 Gibson environments that were rated $4+$ for quality in [1]. For consistency with [1], we use the Gibson dataset [5] for training. Agents are trained from scratch with reinforcement learning using DD-PPO [2] – a decentralized, distributed proximal policy optimization [34] algorithm that is well-suited for GPU-intensive simulator-based training. Each model is trained for 500 million steps on 64 Tesla V100 s. For evaluation, we select the model with best Gibson-val performance. We use the agent architecture from [2] composed of a visual encoder (ResNet50 [35]) and policy network (2-layer LSTM [36] with 512-dim state).

Consistent with prior work, we train agents with RGB and depth sensors. Real-world depth sensors exhibit significant noise and are limited in range. Thus, inspired by the winning entry in the RGB track of the Habitat Challenge 2019 [13], we also test an agent that uses a monocular depth estimator [37] to predict depth from RGB, which is then fed to the navigation model. In total, we train 9 different agents by varying sensor modalities (RGB, Depth, RGB→Predicted Depth) and training simulator configurations (e.g. actuation noise levels). The exact settings are listed in Table I. Note that the simulator parameters used for training these models may differ from the simulator parameters used for testing (θ). Our goal is to span the spectrum of performance at test-time.

[2]Other metrics such as rank correlation can also be used.

TABLE I
AVERAGE PERFORMANCE IN REALITY (COL. 3), CODA AND GIBSON SCENES UNDER HABITAT CHALLENGE 2019 SETTINGS [SLIDING=ON, NOISE=0.0] (COL. 5, COL. 7), AND TEST-SIM [SLIDING=OFF, NOISE=0.0] (COL. 6, COL. 8) ACROSS DIFFERENT TRAIN-SIM CONFIGURATIONS (COL. 1-2)

Sensor	Train-Sim Noise	Train-Sim Sliding	Reality SPL	CODA Chall-Sim SPL	CODA Test-Sim SPL	Gibson Chall-Sim SPL	Gibson Test-Sim SPL
Depth	0.5	off	0.59	0.64	0.58	0.68	0.59
Depth	1.0	off	0.74	0.81	0.70	0.78	0.53
Pred. Depth	0.5	off	0.53	0.37	0.37	0.54	0.40
Pred. Depth	1.0	off	0.66	0.75	0.58	0.64	0.43
RGB	0.5	off	0.33	0.50	0.33	0.56	0.43
RGB	1.0	off	0.44	0.69	0.42	0.58	0.36
Depth	0.0	on	0.64	0.70	0.63	0.66	0.35
Pred. Depth	0.0	on	0.58	0.80	0.44	0.56	0.32
RGB	0.0	on	0.61	0.80	0.64	0.62	0.36

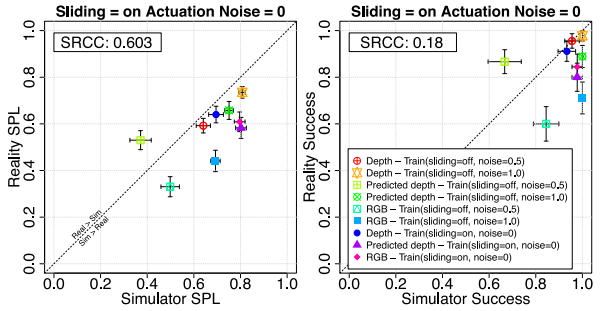


Fig. 5. SRCC_{SPL} (left) and $\text{SRCC}_{\text{Succ}}$ (right) plots for AI Habitat Challenge 2019 test-sim setting in the CODA environment. We note a relatively low correlation between real and simulated performance.

A. Revisiting the AI Habitat Challenge 2019

Fig. 5 plots sim-vs-real performance of these 9 navigation models w.r.t. success rate (right) and SPL [19] (left). Horizontal and vertical bars on a symbol indicate the standard error in simulation and reality respectively.

SRCC_{SPL} is 0.60, which is reasonably high but far from a level where we can be confident about evaluations in simulation alone. Problematically, there are 9 relative ordering reversals from simulation to reality. The success scatterplot (right) shows an even more disturbing trend – nearly all methods (except one) appear to be working *exceedingly* well in simulation with success rates close to 1. However, there is a large dynamic range in success rates in reality. This is summarized by a low $\text{SRCC}_{\text{Succ}}$ of 0.18, suggesting that improvements in performance in simulation are not predictive of performance improvements on a real robot.

Note that other than largely cosmetic adjustments to the robot size, sensor and action space specification, this simulation

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.

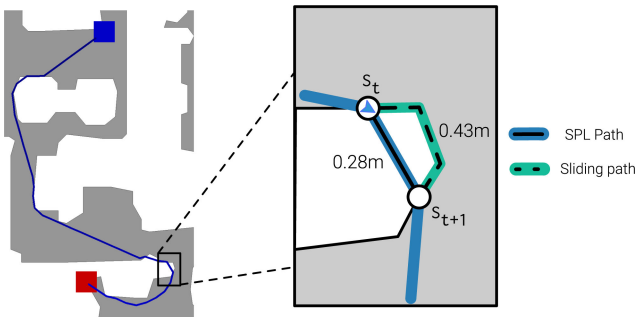


Fig. 6. Sliding behavior leading to 'cheating' agents. At time t, the agent at s_t executes a forward action, and slides along the wall to state s_{t+1}. The resulting straight-line path (used to calculate SPL) goes outside the environment. Gray denotes navigable space while white is non-navigable.

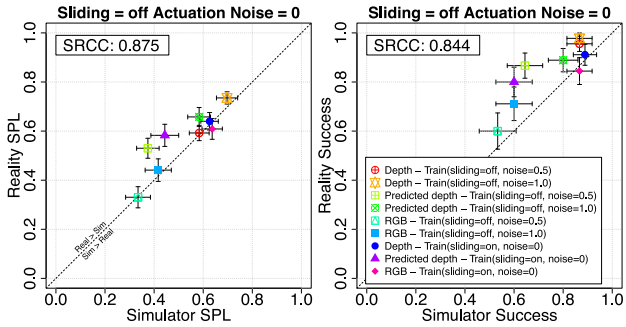


Fig. 7. Optimized SRCC_{SPL} (left) and $\text{SRCC}_{\text{Succ}}$ (right) scatterplots in the CODA environment. Comparing with Fig. 5 we see improvements, indicating better predictivity of real-world performance.

setting is not fundamentally different from the Habitat Challenge 2019. Upon deeper investigation, we discovered that one factor leading to this low sim2real predictivity is due to a 'sliding' behavior in Habitat.

Cheating by sliding: In Habitat-Sim [1], when the agent takes an action that results in a collision, the agent *slides* along the obstacle as opposed to stopping. This behavior is also seen in many other simulators – it is enabled by default in MINOS [9], and Deepmind Lab [6], and is also prevalent in simulators and video game engines that employ physics backend, such as Gibson [5], and AI2 THOR [7]. This is because there is no perfect physics simulator, only approximations; this type of physics allows for smooth human control, but does not accurately reflect real-world physics nor safety precautions the robot platform may employ (*i.e.* stopping on collision). We find that this enables 'cheating' by learned agents. As illustrated by an example in Fig. 6, the agent exploits this sliding mechanism to take an effective path that appears to travel *through non-navigable regions* of the environment (like walls). Let s_t denote agent position at time t, where the agent is already in contact with an obstacle. The agent executes a forward action, collides, and slides along the obstacle to state s_{t+1}. The path taken during this maneuver is far from a straight line, however for the purposes of computing SPL (the metric the agent optimizes), Habitat calculates the Euclidean distance travelled $||s_t - s_{t+1}||_2$. This is equivalent to taking a straight line path between s_t and s_{t+1} that goes outside the navigable regions of the environment (appearing to go through obstacles). On the one hand, the emergence of such exploits is a sign of success of large-scale reinforcement learning – clearly, we are maximizing reward. On the other hand, this is a problem for sim2real transfer. Such policies fail disastrously in the real world where the robot bump sensors force a stop on contact with obstacles. To rectify this issue, we modify Habitat-Sim to disable sliding on collisions. The discovery of this issue motivated our investigation into optimizing simulation parameters.

B. Optimizing Simulation With SRCC

We perform grid-search over simulation parameters – sliding (off vs on) and a scalar multiplier on actuation noise (varying from 0 to 1, in increments of 0.1). We find that sliding off and 0.0 actuation noise lead to the highest SRCC. Fig. 7 shows a remarkable alignment in the SPL scatter-plot (left) – nearly all models lie close to the diagonal, suggesting that we fairly accurately predict how a model is going to perform on the robot by testing in simulation. Recall that the former takes 40.5 hours and significant human effort, while the latter is computed in under 1 minute. SRCC_{SPL} improves from 0.603 (Fig. 5) to 0.875 (0.272 gain). $\text{SRCC}_{\text{Succ}}$ shows an even stronger improvement – from 0.18 to 0.844 (0.664 gain)!

From Table I, we can see that the best performance in reality (Reality SPL) is achieved by row 2, which is confidently predicted by CODA Test-Sim SPL (col 4) by a strong margin. The fact that no actuation noise is the optimal setting suggests that our chosen actuation noise model (from PyRobot [18]) may not reflect conditions in reality.

To demonstrate the capabilities of our models in navigating cluttered environments, we also evaluate performance on the Gibson dataset, which contains 572 scanned spaces (apartments, multi-level homes, offices, houses, hospitals, gyms), containing furniture (chairs, desks, sofas, tables), for a total of 1447 floors. We use the 'Gibson-val' navigation episodes from Savva *et al.* [1]. We observe a similar drop between performance between sliding on (Chall-Sim col 5, 7 in Table I) vs sliding off (Test-Sim col 4,6 in Table I). This further suggests that the participants in the Habitat Challenge 2019 would not achieve similar performance on a real robot.

We also implement a wall-following oracle, and compare its performance to our learned model (Depth, actuation noise=0.5, sliding=off). The oracle receives full visibility of the environment via a top-down map of the environment. Note that this is significantly stronger input than our learned models (which operate on egocentric depth frames). The oracle navigates to the goal by following a straight path towards the goal and follows walls upon coming in contact with obstacles. We endow the oracle with perfect wall-following (*i.e.* it follows the wall along the shortest direction to the goal), and it never actually collides or gets stuck on obstacles. This oracle achieves 0.46 SPL on Gibson-val, compared to 0.59 SPL achieved by our model under the Gibson Test-Sim setting. For episodes longer than 10 m, the oracle performance drops to 0.05 SPL, compared to our model, which achieves a 0.23 SPL. These experiments show

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.

the need for a learning based approach in complex and cluttered environments.

VI. Conclusion

We introduce the Habitat-PyRobot Bridge (HaPy) library that allows for seamless deployment of visual navigation models. Using Matterport scanning, the Habitat stack, the HaPy library, and LoCoBot we benchmark the correlation between reality and simulation performance with the SRCC metric. We find that naive simulation parameters lead to low correlation between performance in simulation and performance in reality. We then optimize the simulation parameters for SRCC and obtain high predictivity of real world performance without running new evaluations in the real world. We hope that the infrastructure we introduce and the conceptual framework of optimizing simulation for predictivity will enable sim2real transfer in a variety of navigation tasks.

Acknowledgment

We thank the reviewers for their helpful suggestions. We are grateful to Kalyan Alwala, Dhiraj Gandhi and Wojciech Galuba for their help and support. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the U.S. Government, or any sponsor.

Licenses for referenced datasets:

Gibson: https://storage.googleapis.com/gibson_material/Agreement%20GDS%2006-04-18.pdf

Matterport3D: http://kaldir.vc.in.tum.de/matterport/MP_TOS.pdf

References

[1] M. Savva *et al.*, "Habitat: A platform for embodied AI research," in *Proc. Int. Conf. Comput. Vision*, 2019, pp. 1–4, 6, 7.

[2] E. Wijmans *et al.*, "DD-PPO: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1, 6.

[3] S. Gupta, J. Davidson, S. Levine, R. Sukthankar, and J. Malik, "Cognitive mapping and planning for visual navigation," in *Proc. Comput. Vision Pattern Recognit. Conf.*, 2017, p. 1.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annu. Conf. Robot Learn.*, 2017, pp. 1–16.

[5] F. Xia, A. R. Zamir, Z. He, A. Sax, J. Malik, and S. Savarese, "Gibson env: Real-world perception for embodied agents," in *Proc. Comput. Vision Pattern Recognit. Conf.*, 2018, pp. 1–7.

[6] C. Beattie *et al.*, "Deepmind lab," 2016, *arXiv:1612.03801*.

[7] E. Kolve *et al.*, "AI2-THOR: An interactive 3D environment for visual AI," 2017, *arXiv:1712.05474*.

[8] F. Sadeghi and S. Levine, "CAD2RL: real single-image flight without a single real image," in *Robotics: Science and Systems XIII*, Cambridge, Massachusetts, USA, Jul. 2017.

[9] M. Savva, A. X. Chang, A. Dosovitskiy, T. Funkhouser, and V. Koltun, "MINOS: Multimodal indoor simulator for navigation in complex environments," 2017, *arXiv:1712.03931*.

[10] P. Anderson *et al.*, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proc. Comput. Vision Pattern Recognit. Conf.*, 2018, pp. 1, 5.

[11] A. Das, S. Datta, G. Gkioxari, S. Lee, D. Parikh, and D. Batra, "Embodied Question Answering," in *Proc. Comput. Vision Pattern Recognit. Conf.*, 2018, p. 1.

[12] "Visual Learning and Embodied Agents in Simulation Environments @ ECCV 2018," 2018. [Online]. Available: https://eccv18-vlease.github.io/

[13] "Habitat Challenge 2019 @ Habitat Embodied Agents Workshop. CVPR 2019," 2019. [Online]. Available: https://aihabitat.org/challenge/2019/

[14] "CARLA autonomous driving challenge @ CVPR 2019," 2019. [Online]. Available: https://carlachallenge.org/

[15] "Robothor challenge @ CVPR 2020," 2020. [Online]. Available: https://ai2thor.allenai.org/robothor/challenge/

[16] J. Lehman *et al.*, "The surprising creativity of digital evolution: A collection of anecdotes from the evolutionary computation and artificial life research communities," 2018, *arXiv:1803.03453*.

[17] B. Baker *et al.*, "Emergent tool use from multi-agent autocurricula," 2019, *arXiv:1909.07528*.

[18] A. Murali *et al.*, "Pyrobot: An open-source robotics framework for research and benchmarking," 2019, *arXiv:1906.08236*.

[19] P. Anderson *et al.*, "On Evaluation of Embodied Navigation Agents," 2018, *arXiv:1807.06757*.

[20] "Locobot: An open source low cost robot," 2019. [Online]. Available: https://locobot-website.netlify.com/

[21] Y. Zhu *et al.*, "Target-driven visual navigation in indoor scenes using deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2017, p. 2.

[22] S. Gupta, J. Davidson, S. Levine, R. Sukthankar, and J. Malik, "Cognitive mapping and planning for visual navigation," in *Proc. Comput. Vision Pattern Recognit. Conf.*, 2017, p. 2.

[23] D. Gordon, A. Kadian, D. Parikh, J. Hoffman, and D. Batra, "Splitnet: Sim2sim and task2task transfer for embodied visual navigation," in *Proc. Int. Conf. Comput. Vision*, 2019, p. 2.

[24] E. Krotkov *et al.*, "The darpa robotics challenge finals: results and perspectives," *J. Field Robot.*, vol. 34, no. 2, pp. 229–240, 2017.

[25] D. Gandhi, L. Pinto, and A. Gupta, "Learning to fly by crashing," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2017, pp. 3948–3955.

[26] M. Müller, A. Dosovitskiy, B. Ghanem, and V. Koltun, "Driving policy transfer via modularity and abstraction," 2018, *arXiv:1804.09364*.

[27] J. Tan *et al.*, "Sim-to-real: Learning agile locomotion for quadruped robots," 2018, *arXiv:1804.10332*.

[28] Y. Chebotar *et al.*, "Closing the sim-to-real loop: Adapting simulation randomization with real world experience," in *Proc. Int. Conf. Robot. Autom.* 2019, pp. 8973–8979.

[29] J. Hwangbo *et al.*, "Learning agile and dynamic motor skills for legged robots," 2019, *arXiv:1901.08652*.

[30] M. Quigley *et al.*, "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Softw.*, 2009, vol. 3, p. 3.

[31] D. Mishkin, A. Dosovitskiy, and V. Koltun, "Benchmarking classic and learned navigation in complex 3D environments," 2019, *arXiv:1901.10915*.

[32] N. Kojima and J. Deng, "To learn or not to learn: Analyzing the role of learning for navigation in virtual environments," 2019, *arXiv:1907.11770*.

[33] S. Kohlbrecher, J. Meyer, O. von Stryk, and U. Klingauf, "A flexible and scalable slam system with full 3d motion estimation," in *Proc. IEEE Int. Symp. Safety, Secur. Rescue Robot.*, 2011, pp. 155–160.

[34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[35] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. Comput. Vision Pattern Recognit. Conf.*, 2016, p. 6.

[36] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[37] D. Wofk, F. Ma, T.-J. Yang, S. Karaman, and V. Sze, "FastDepth: Fast monocular depth estimation on embedded systems," in *Proc. ICRA*, 2019, p. 6.

Authorized licensed use limited to: Universidad Federal de Pernambuco. Downloaded on September 02,2026 at 04:16:15 UTC from IEEE Xplore. Restrictions apply.